

T3: Web MVP Visual-first AI Website Generator

Цель

Собрать минимальное веб-приложение, которое доказывает главный пайплайн:

**бриф пользователя -> визуальный референс hero-блока -> описание дизайна -> frontend-код
-> скриншот результата -> сравнение с референсом -> отчет по отличиям**

Это не полноценный SaaS и не конструктор сайтов. Главная цель MVP - проверить, может ли система стабильно превращать утвержденный визуальный референс в похожий frontend-код.

1. Формат продукта

Нужно сделать веб-приложение с простым интерфейсом:

1. Пользователь открывает страницу приложения.
2. Вводит текстовый бриф.
3. Нажимает кнопку запуска генерации.
4. Система создает новый запуск проекта.
5. Пользователь видит статус выполнения пайплайна.
6. После завершения пользователь видит:
 - исходный визуальный референс;
 - сгенерированный сайт в preview;
 - desktop/mobile screenshots;
 - отчет визуального сравнения;
 - ссылку/кнопку для скачивания сгенерированного frontend-кода.

В первой версии не нужен сложный UI. Интерфейс должен быть рабочим, понятным и достаточным для проверки пайплайна.

2. Технологический стек

Frontend

- React
- TypeScript
- Vite
- React Router
- TanStack Query
- Axios
- SCSS Modules
- React Markdown для отображения `visual-report.md`
- React Toastify или аналог для уведомлений

Архитектуру frontend нужно брать по примеру проекта:

```
C:\Development\frontend\agent_web
```

Важно: не переносить бизнес-логику, авторизацию, подписки и лишние сущности из `agent_web`.

Нужно взять именно подход к организации кода:

- `src/app` - инициализация приложения, роутер, провайдеры;
- `src/features` - продуктовые фичи и страницы;
- `src/shared` - переиспользуемые API-клиенты, UI-kit, widgets, hooks, styles, lib;
- CSS Modules/SCSS рядом с компонентами;
- отдельные API service modules в `shared/api/services`;
- lazy routes через React Router;
- общие константы маршрутов в `shared/model/routes.ts`;
- общий `queryClient`;
- общий `axiosInstance`.

Backend

- NestJS
- TypeScript
- TypeORM
- PostgreSQL
- BullMQ или другой job queue для фоновых задач
- Redis для очереди, если используется BullMQ

Генерация и автоматизация

- AI image generation API для создания визуального референса
- AI vision/text model для анализа изображения и генерации описания дизайна
- AI code generation для создания frontend-кода
- Playwright для запуска preview и создания скриншотов
- pixelmatch или похожий инструмент для diff.png
- sharp/pngjs для работы с изображениями

DevOps для локального запуска

- Docker Compose для PostgreSQL и Redis
- `.env` для конфигурации API ключей и путей
- npm workspaces или pnpm workspaces для monorepo

3. Структура проекта

```
project/  
  apps/
```

```
web/  
  src/  
    app/  
      app.tsx  
      main.tsx  
      components/  
        AppRoutes.tsx  
      model/  
        router.tsx  
        providers.tsx  
  features/  
    runs/  
      pages/  
        RunsList/  
          runs-list.page.tsx  
          runs-list.module.scss  
        RunDetails/  
          run-details.page.tsx  
          run-details.module.scss  
      components/  
        BriefForm/  
        RunStatusBadge/  
        ArtifactPreview/  
        VisualReport/  
        LogsPanel/  
      hooks/  
      model/  
    not-found/  
      pages/  
shared/  
  api/  
    axiosInstance.ts  
    queryClient.ts  
    services/  
      runs/  
        runs.api.ts  
        hooks.ts  
        types.ts  
        index.ts  
  components/  
    ErrorBoundary/  
  hooks/  
  kit/  
    Buttons/  
    Inputs/  
    UI/  
  lib/  
    download/  
    error/  
    lazy-import.ts  
  model/  
    config.ts  
    routes.ts
```

```
    styles/  
      main.scss  
      reset.scss  
      variables.scss  
    widgets/  
      Layout/  
      FullScreenLoader/  
      FullErrorScreen/  
package.json  
vite.config.ts  
tsconfig.json  
  
api/  
  src/  
    main.ts  
    app.module.ts  
    config/  
    database/  
    runs/  
    generation/  
    storage/  
    screenshot/  
    visual-qa/  
package.json  
nest-cli.json  
tsconfig.json  
  
generated/  
  runs/  
    run-001/  
      reference/  
        hero-reference.png  
  
    design/  
      design-description.md  
      design-tokens.json  
  
    code/  
      frontend-project/  
  
    screenshots/  
      rendered-desktop.png  
      rendered-mobile.png  
  
    qa/  
      reference-validation.md  
      visual-report.md  
      diff.png  
      build-error.md  
  
project-spec.json  
status.json
```

```
packages/  
  shared/  
    src/  
      types/  
      constants/  
  
docker-compose.yml  
package.json  
.env.example  
README.md
```

4. Основной пользовательский сценарий

4.1 Создание запуска

Пользователь вводит бриф в textarea.

Пример брифа:

Сделай первый экран лендинга для AI-сервиса финансовой аналитики.

Стиль:

- темный
- дорогой
- современный
- крупная типографика
- фиолетово-синие акценты
- карточка продукта справа

Текст:

Заголовок: AI-аналитика для финансовых команд

Описание: Получайте инсайты, прогнозы и отчеты быстрее без ручной рутины.

Основная кнопка: Начать бесплатно

Вторая кнопка: Смотреть демо

Frontend отправляет запрос:

```
POST /api/runs  
Content-Type: application/json  
  
{  
  "brief": "..."  
}
```

Backend создает запись **Run**, папку **generated/runs/run-XXX/**, сохраняет бриф и ставит job в очередь.

5. Сущности БД

Run

Хранит один запуск генерации.

Поля:

- `id: uuid`
- `runNumber: number`
- `slug: string` - например `run-001`
- `brief: text`
- `status: RunStatus`
- `currentStep: string | null`
- `score: number | null`
- `errorMessage: text | null`
- `createdAt: timestamp`
- `updatedAt: timestamp`

RunArtifact

Хранит ссылки на файлы, созданные в процессе запуска.

Поля:

- `id: uuid`
- `runId: uuid`
- `type: ArtifactType`
- `path: string`
- `mimeType: string | null`
- `createdAt: timestamp`

RunLog

Хранит события пайплайна.

Поля:

- `id: uuid`
- `runId: uuid`
- `level: info | warning | error`
- `message: text`
- `metadata: jsonb | null`
- `createdAt: timestamp`

6. Статусы запуска

```
type RunStatus =  
  | 'queued'  
  | 'running'  
  | 'reference_failed'  
  | 'build_failed'  
  | 'visual_failed'  
  | 'needs_manual_review'  
  | 'completed'  
  | 'failed';
```

7. Backend API

Создать запуск

POST /api/runs

Request:

```
{  
  "brief": "string"  
}
```

Response:

```
{  
  "id": "uuid",  
  "slug": "run-001",  
  "status": "queued"  
}
```

Получить список запусков

GET /api/runs

Получить запуск

GET /api/runs/:id

Response должен включать:

- основные поля запуска;
- текущий статус;
- список артефактов;
- последние логи;
- visual score, если есть.

Получить файл артефакта

```
GET /api/runs/:id/artifacts/:artifactId
```

Скачать сгенерированный код

```
GET /api/runs/:id/download-code
```

Возвращает `.zip` с папкой `code/frontend-project`.

8. Frontend UI

Архитектурные правила

Frontend должен быть организован по образцу `C:\Development\frontend\agent_web`:

- `app/model/router.tsx` отвечает за `createBrowserRouter` и lazy routes;
- `app/model/providers.tsx` подключает `QueryClientProvider`, toast provider и будущие глобальные провайдеры;
- страницы лежат внутри feature-модулей, например `features/runs/pages/RunDetails/run-details.page.tsx`;
- крупные части страницы выносятся в `features/runs/components`;
- сетевые запросы лежат в `shared/api/services/runs`;
- базовый axios client лежит в `shared/api/axiosInstance.ts`;
- TanStack Query hooks лежат рядом с API service, например `shared/api/services/runs/hooks.ts`;
- общие UI-компоненты лежат в `shared/kit`;
- layout, loader и error screen лежат в `shared/widgets`;
- стили компонентов пишутся через `*.module.scss`;
- глобальные стили лежат в `shared/styles`;
- импорты должны идти через alias `@/`.

Минимальные frontend routes:


```
export const ROUTES = {
  RUNS: '/',
  RUN_DETAILS: '/runs/:runId'
} as const;
```

Главная страница

Должна содержать:

- textarea для брифа;
- кнопку **Generate**;
- список последних запусков;
- статус последнего запуска.

Страница запуска

Должна содержать:

- статус пайплайна;
- текущий шаг;
- логи выполнения;
- изображение **hero-reference.png**;
- preview/screenshot результата;
- mobile screenshot;
- **diff.png**;
- markdown-отчет **visual-report.md**;
- кнопку скачать код.

Обновление статуса можно сделать polling через TanStack Query каждые 2-3 секунды. WebSocket в первой версии не обязателен.

9. Pipeline генерации

Pipeline выполняется на backend в фоновой job.

Шаг 1 - подготовка брифа

Сырой бриф нужно превратить во внутреннее структурированное описание.

Файл:

```
generated/runs/run-001/project-spec.json
```

Пример:

```
{
  "siteType": "landing",
  "sectionType": "hero",
  "style": ["dark", "premium", "modern", "saas"],
  "audience": "finance teams",
  "requiredElements": [
    "headline",
    "description",
    "primaryButton",
    "secondaryButton",
    "productCard"
  ],
  "copy": {
    "headline": "AI-аналитика для финансовых команд",
    "description": "Получайте инсайты, прогнозы и отчеты быстрее без ручной рутины.",
    "primaryButton": "Начать бесплатно",
    "secondaryButton": "Смотреть демо"
  },
  "visualPreferences": [
    "dark background",
    "purple-blue accents",
    "large typography",
    "product card on the right"
  ]
}
```

Шаг 2 - генерация визуального референса

Нужно сгенерировать изображение hero-блока.

Требования:

- только один hero-блок;
- без браузерной рамки;
- без лишних секций;
- размер примерно 1440x900;
- дизайн должен быть реализуемым через frontend;
- без слишком сложных 3D-объектов;
- текст читаемый;
- структура понятная.

Файл:

generated/runs/run-001/reference/hero-reference.png

Шаг 3 - проверка визуального референса

Проверить:

- есть ли понятная сетка;
- читается ли текст;
- понятно ли расположение элементов;
- нет ли слишком сложной графики;
- можно ли повторить дизайн через CSS/Tailwind;
- нет ли лишних блоков;
- соответствует ли картинка брифу.

Если референс плохой:

- создать `generated/runs/run-001/qa/reference-validation.md`;
- поставить статус `reference_failed`;
- остановить pipeline.

Шаг 4 - описание дизайна

Создать:

```
generated/runs/run-001/design/design-description.md
```

В описании должны быть:

- фон;
- сетка;
- типографика;
- кнопки;
- карточки;
- декоративные элементы;
- адаптивное поведение.

Шаг 5 - дизайн-токены

Создать:

```
generated/runs/run-001/design/design-tokens.json
```

Пример:

```
{
  "colors": {
    "background": "#050816",
    "textPrimary": "#FFFFFF",
    "textSecondary": "#A7B0C0",
```

```

    "accent": "#7C3AED",
    "surface": "rgba(255,255,255,0.06)"
  },
  "layout": {
    "containerWidth": "1200px",
    "sectionPaddingY": "96px",
    "sectionPaddingX": "32px",
    "columns": 2
  },
  "typography": {
    "headlineSize": "72px",
    "headlineWeight": 700,
    "bodySize": "18px"
  },
  "components": {
    "buttonRadius": "999px",
    "cardRadius": "24px"
  }
}

```

Шаг 6 - генерация frontend-кода

Backend должен сгенерировать минимальный frontend-проект.

Стек сгенерированного проекта:

- Next.js;
- React;
- TypeScript;
- Tailwind CSS.

Путь:

```
generated/runs/run-001/code/frontend-project/
```

Минимальная структура:

```

frontend-project/
  app/
    page.tsx
    layout.tsx
    globals.css

  components/
    Hero.tsx

  public/

```

```
package.json  
tailwind.config.ts  
tsconfig.json
```

Требования:

- одна страница;
- один hero-блок;
- текст соответствует брифу;
- композиция повторяет reference image;
- без сторонних UI-библиотек;
- без лишних секций.

Шаг 7 - проверка сборки

Backend должен:

1. Установить зависимости в сгенерированном проекте.
2. Запустить build.
3. Если build упал, создать:

```
generated/runs/run-001/qa/build-error.md
```

4. Попробовать исправить только технические ошибки.

Максимум попыток исправления сборки: 2.

Если исправить не удалось, поставить статус **build_failed**.

Шаг 8 - preview и скриншоты

После успешной сборки backend запускает preview/dev server сгенерированного проекта и делает скриншоты через Playwright.

Файлы:

```
generated/runs/run-001/screenshots/rendered-desktop.png  
generated/runs/run-001/screenshots/rendered-mobile.png
```

Размеры:

- desktop: 1440x900;
- mobile: 390x844.

Шаг 9 - визуальное сравнение

Сравнить:

```
generated/runs/run-001/reference/hero-reference.png
generated/runs/run-001/screenshots/rendered-desktop.png
```

Проверить:

- похожа ли композиция;
- совпадает ли расположение заголовка;
- похожи ли размеры текста;
- совпадает ли цветовая схема;
- похожи ли кнопки;
- похожа ли карточка справа;
- не потерялись ли элементы;
- не появился ли лишний мусор;
- похож ли общий визуальный стиль.

Создать:

```
generated/runs/run-001/qa/visual-report.md
generated/runs/run-001/qa/diff.png
```

Формат отчета:

```
# Visual QA Report

## Общая оценка

7/10

## Что совпало

- Цветовая схема близка.
- Композиция в целом похожа.
- Основные элементы на месте.

## Что не совпало

- Заголовок слишком маленький.
- Правая карточка расположена ниже, чем в референсе.
- Фон недостаточно глубокий.

## Критичные проблемы

- Нет.
```

Рекомендации для исправления

1. Увеличить заголовок на 15-20%.
2. Поднять правую карточку выше.
3. Усилить фоновый градиент.

Шаг 10 - финальный статус

Обновить:

```
generated/runs/run-001/status.json
```

Возможные финальные статусы:

- `completed;`
- `reference_failed;`
- `build_failed;`
- `visual_failed;`
- `needs_manual_review;`
- `failed.`

10. Что не нужно делать в первой версии

В MVP не нужно:

- регистрация;
- авторизация;
- роли пользователей;
- оплата;
- личный кабинет;
- история проектов между пользователями;
- drag-and-drop редактор;
- Figma export;
- deploy на Vercel;
- генерация полного сайта;
- многостраничные сайты;
- CMS;
- формы;
- ручная адаптивная кастомизация;
- красивый сложный UI;
- WebSocket, если хватает polling;
- автоматический visual repair loop.

11. Критерии готовности MVP

MVP считается готовым, если:

1. Приложение запускается локально через README.
2. Frontend открывается в браузере.
3. Пользователь может отправить бриф.
4. Backend создает **Run** в PostgreSQL.
5. Backend выполняет pipeline в фоне.
6. Все артефакты сохраняются в **generated/runs/run-XXX/**.
7. UI показывает статус запуска.
8. UI показывает reference image, screenshots, diff и markdown-отчет.
9. Сгенерированный frontend-код можно скачать.
10. Ошибки build/reference validation сохраняются в отчетах.

12. Следующий этап, но не делать сейчас

После MVP можно добавить:

- автоматический repair loop;
- генерацию нескольких секций: Hero, Features, CTA;
- ZIP export с настройками;
- историю запусков с фильтрами;
- WebSocket progress updates;
- multi-reference generation;
- выбор лучшего варианта;
- ручное approve/reject для reference image;
- пользовательские аккаунты;
- SaaS-упаковку.